TicketFlow

Design and build an event ticket booking system

Design and build, on your own, the backend of a platform that sells tickets for live events at venues with seat maps. When a popular event goes on sale, thousands of customers compete for the same seats at the same moment. The system must never sell a seat twice, never get money wrong, and keep working under load and when parts fail. You make every design decision, and you are expected to explain and defend each one.

Functional requirements

- **Venues and events:**
  - Venues have seat maps of up to 20,000 seats, made up of sections, rows of numbered seats, and standing areas with a capacity.
  - Events have types, and each type has its own attributes (for example, performers for concerts and teams for sports). Staff can add new attributes without a redeploy.
  - Anyone can browse, filter and search events.
- **Seat management:** organizers set prices per section and can block seats. Every change is audited.
- **Bookings:**
  - Customers book specific seats, a number of "best available" seats next to each other, or standing places.
  - Seats are held until payment. Unpaid holds expire after 15 minutes and release their seats.
  - Customers can cancel up to 48 hours before the event.
- **Payments:** payments go through an external payment provider, which you also build as a fake that can be slow, fail, or lose its replies. Support staff can issue full or partial refunds. Every money movement is recorded and reconciled every day.
- **On-sales:** tickets become available at a scheduled time, optionally with a limited high-demand batch, and each customer can buy at most a set number of tickets per event.
- **Event cancellation:** every paid booking is refunded automatically, and all of the event's tickets stop being valid.
- **Tickets and check-in:** tickets carry codes that can be verified and can't be guessed. Each ticket gets in only once, even when it is scanned at several gates at the same moment, and gate devices may send the same scan twice.
- **History and notifications:** each booking has a full timeline, and each customer has an activity feed covering the last 90 days. Customers are notified of key events and can turn off the optional ones.
- **Users:** customers, event organizers (who manage only their own events) and support agents. Everyone sees only what their role allows.

Non-functional requirements

- **Correctness:**

    - A seat is never sold twice, even with 1,000 buyers competing for the same 10 seats.
    - No booking is charged twice or refunded twice.
    - An accepted booking is never lost.
- **Scale:**
    - 100,000 events and millions of history events.
    - At 200 requests per second, 95% of reads complete within 200 ms and 95% of writes within 400 ms.
- **Resilience:** the system keeps working when any single instance, the message broker, the cache or the payment provider fails or slows down, and it recovers by itself.
- **Security:** login through an identity provider, rate limiting, and no personal data, ticket codes or secrets in logs.
- **Operability:**
    - Metrics, dashboards, alerts, centralized logs and end-to-end tracing.
    - Deployments with zero downtime.

Required technologies

Must use	Java 21+, Spring Boot, Maven **Oracle, MongoDB, Cassandra** (each must own real data) Kafka, Redis, OpenFeign, Spring Cloud Gateway Prometheus, Grafana, Splunk GitHub Actions, Jenkins, GitLab CI, Bitbucket Pipelines Kubernetes, OpenShift
Must NOT use	PostgreSQL
Out of scope	User interface (APIs only) and AI features.

How and where you use each technology is part of your design.

What to deliver

- A design document with architecture diagrams, your data model, API and message contracts, and a decision record for every significant choice.
- Working code in your own repository, with automated tests.
- Evidence that the non-functional requirements are met: load test results and failure experiments.
- A README from which anyone can run the system, plus a short demo.
- A design review with the other developer, run as a system design interview.